The Monday After the 3 AM Page, Continued

Lecture 4 · An AI pair programmer, then the debugger

GitHub Copilot · logs · breakpoints · the Debug Console

Slides, demos and labs: `lectures/04-ai-debugging-continued/` (links to Lecture 3)

Today

Time	What	Where to find it
3 min	Where we left ORD-317	
10 min	**Act 5:** three ways to work with AI	Lecture 3 · Demo 2, Step 5
20 min	**Act 6:** debugging ORD-317	Lecture 3 · Demo 3
7 min	**Three Mentimeter questions** during the demos	menti.com · code 6357 2204
8 min	**Attendance form** and **anonymous feedback**	QR codes at the end

Everything I type today is written up, step by step, in the Lecture 3 demo guides. Follow along on your laptop, or just watch.

Previously, on Mastering CS…

- **ORD-317:** Sam Patel (Silver card) paid **£5.95**. Grace in Finance says **£7.29**.
- You reproduced it: the loyalty discount appears **three times**.
- You found a suspicious loop (Priya, ORD-301) and renamed `calc` for Mei (`F2`).
- Smoke tests: `17 passed, 200 deselected, 1 xfailed`.

> **Mei** is happy. **Sam** is still being undercharged.
> Clear code first. Now: fix the bug.

By the end of today you can…

1. Choose between **autocomplete**, **inline chat** and **chat** for a task
2. Explain why you must **check** what an AI produces, and how
3. Follow a **debugging routine**: reproduce, hypothesise, observe, fix, verify
4. Use **logs**, a **conditional breakpoint**, **Variables**, **Watch** and the **Debug Console** to find a bug
5. Know your options when there's **no IDE** (`pdb`)

Act 5

"Get a pair programmer"

Three ways to work with AI in the IDE

Three ways to work with AI

Mode	How (Windows/Linux · macOS)	Good for
Autocomplete	Just type; grey text appears; `Tab` accepts	Finishing code you've started
Inline chat	Select code, then `Ctrl+I` · `Cmd+I`	Changing **one specific piece** of code; you review a **diff**
Chat	Chat view: `Ctrl+Alt+I` · `Ctrl+Cmd+I`	**Ask:** questions, no edits · **Agent:** multi-file tasks

Since Copilot launched in 2021, AI assistants have become a normal part of programming: **84%** of developers use or plan to use them (Stack Overflow, 2025).

Demo: 1. Autocomplete, steered by a name and a docstring

At the end of `services/receipts.py`, type, then press `Enter` and wait:

```
def discount_summary(order: Order) -> str:
    """One line per discount, e.g. "LOYALTY −£0.81", in the order applied."""
```

- `Tab` accepts · `Alt+]` · `Opt+]` shows other suggestions
- A precise **name**, **type hints** and a **docstring** steer the AI better than throwaway comments
- **Read it before you keep it**

When the AI gets it wrong, the tools notice

In rehearsal, Copilot suggested:

```
if d.amount > 0
```

Pylance underlined it at once:

> Operator ">" not supported for types "Money" and "Literal[0]"

The team's `Money` type can't be compared with a plain number (ADR 0002).
The type checker caught the AI's mistake. That's why the team turns it on.

Demo: 2. Inline chat, then 3. Ask about the codebase

Inline chat: select `discount_summary`, press `Ctrl+I` · `Cmd+I`, type:

```
Handle an order with no discounts by returning 'No discounts'.
```

Read the **diff** (removed and added lines), then **Keep** or **Undo**.

Chat, in Ask mode (Ask answers questions without editing files):

```
Where is the loyalty discount applied in this project, and why could a
loyalty customer be undercharged when their basket has several categories?
```

Mentimeter question 1 of 3

You want Copilot to change **one function you have selected**, so that it returns `'No discounts'` for an empty order. Which way of working with AI fits best?

1. Autocomplete
2. Inline chat
3. Chat in **Ask** mode
4. Chat in **Agent** mode



menti.com
code **6357 2204**

Question 1: the answer is 2, inline chat

Option	Why
Autocomplete	Finishes what you're typing; you can't describe a change
2. Inline chat (correct)	**One selected piece of code; you review a diff**
Chat, Ask mode	Answers questions; doesn't edit your files
Chat, Agent mode	Works across many files: too much power for a two-line change

> Rule of thumb: the **smallest** AI tool that does the job is the easiest one to check.

AI is a lead, not an oracle

Study	Finding
Stack Overflow, 2025	**46%** don't trust AI accuracy; top frustration (66%): "almost right, but not quite"
METR randomised trial, 2025 (preprint)	Experienced developers were **19% slower** with AI, but felt **20% faster**
Cui et al., Management Science, 2026 (4,867 developers)	**+26%** tasks completed; juniors gained most

> Felt 20% faster, was 19% slower. Like me at the gym.

`AGENTS.md` = the team's rules for the AI · **never paste secrets** · **Jordan:** "Great. Now **prove it**."

Act 6

"Why are loyalty customers undercharged?"

Debugging

The five stages of print debugging

```
print("here")
print("here 2")
print("HERE??")
print(running)            # Money(amount=Decimal('9.50'), currency='GBP') ...ok?
print("why is this happening")
```

> "The most effective debugging tool is still careful thought, coupled with judiciously placed print statements." (Brian Kernighan)

Today: what comes after the print statements.

A routine, not luck

Stage	Question	ORD-317
1. Reproduce	Can I make it happen on demand?	The CLI quote shows £5.95
2. Hypothesise	What could cause exactly this?	Loyalty runs once per category
3. Observe	What is the program actually doing?	Logs, then the debugger
4. Fix	The smallest change that removes the cause	Move loyalty out of the loop
5. Verify	Fixed, and nothing else broken?	The CLI says £7.29

Novices are better at **fixing** bugs than **finding** them (Fitzgerald et al., 2008). Today is about finding.

Demo: ask the logs first

macOS / Linux (PowerShell version in the Demo 3 guide):

```
NOVASTACK_LOG_LEVEL=DEBUG uv run novastack quote \
    data/samples/meal_deal_plus_drink.json --at 2026-10-07T12:30
```

```
applied LOYALTY: -0.52 (running total 5.20 -> 4.68)
applied LOYALTY: -0.68 (running total 6.78 -> 6.10)
applied LOYALTY: -0.95 (running total 9.50 -> 8.55)
applied MEALDEAL: -2.60 (running total 8.55 -> 5.95)
```

Discuss: two things are wrong. What are they?

Demo: stop at exactly the right moment

1. In `pricing/calculator.py`, right-click the margin by `if self._loyalty.applies(ctx):`
2. **Add Conditional Breakpoint…** → Expression:

```
category == "drink"
```

3. **Run and Debug** (`Ctrl+Shift+D` · `Cmd+Shift+D`) → **CLI: quote the sample meal deal** → `F5` (Mac: `Fn+F5`)

Variables: `running` = **£9.50** · `applied` = **2** discounts, both `LOYALTY`

Mentimeter question 2 of 3

We're paused at the breakpoint `category == "drink"` (the **third** category). `running` is £9.50 and `applied` already holds **2** `LOYALTY` discounts. What does this tell us?

1. The meal deal discount was applied twice
2. The loyalty discount is applied once **per category**, inside the loop
3. The breakpoint condition is wrong
4. Nothing yet: we need more print statements

menti.com
code **6357 2204**

Question 2: the answer is 2, once per category

- We are on the **third** category and loyalty has **already** been applied **twice**
- When we continue, it is applied a **third** time
- That's our hypothesis, now **confirmed with evidence**, not a guess

> The meal deal hasn't been applied at all yet, which is the second bug: the architecture says **promotions first, then loyalty once**.

Demo: ask questions while paused

Debug Console (`Ctrl+Shift+Y` · `Cmd+Shift+Y`): Python **inside the paused program**. One line at a time:

Type	Answer
`len(applied)`	`2`
`[d.code for d in applied]`	`['LOYALTY', 'LOYALTY']`
`[str(d.amount.amount) for d in applied]`	`['0.52', '0.68']`

Step: `F10` over · `F11` into · `Shift+F11` out · `F5` continue · `Shift+F5` stop
Mac laptops: hold **`Fn`** (`Fn+F10`), or use the debug toolbar.

Demo: no pausing, no editing, no IDE

Tool	When	Example
Logpoint	See every pass, without stopping or editing code	`{category}: running £{running.amount}`
`breakpoint()` + pdb	On a server over SSH, no VS Code	`p category` · `n` · `c` · `q`
AI, given evidence	Explain logs, stack traces, cryptic errors	Paste the 4 DEBUG lines, ask why

Never commit `breakpoint()`: production would freeze, waiting at a `(Pdb)` prompt.

> AI is **very good at finding** bugs from evidence and **less good at fixing** them. Give it evidence, not "it's broken".

Mentimeter question 3 of 3

You want to see the running total on **every pass** of the loop, **without pausing** the program and **without changing** the code. Which tool?

1. `print()`
2. A conditional breakpoint
3. A logpoint
4. `breakpoint()` with pdb



menti.com
code **6357 2204**

Question 3: the answer is 3, a logpoint

Option	Pauses?	Edits the code?
`print()`	No	**Yes**, and you must remember to delete it
Conditional breakpoint	**Yes**, when the condition is true	No
3. Logpoint (correct)	**No**	**No**
`breakpoint()` with pdb	**Yes**	**Yes**

> A logpoint is a print statement you never have to delete, because it was never in the code.

Fix it, then verify with the same evidence

Move the loyalty discount **out of the loop**, after promotions:

```
        for rule in self._promotions:
            if rule.applies(ctx):
                running = self._apply(rule, ctx, running, applied)

        # Loyalty applies once, to the whole promoted total (ORD-317).
        if self._loyalty.applies(ctx):
            running = self._apply(self._loyalty, ctx, running, applied)
```

Same command again: meal deal **–£2.60**, loyalty **–£0.81**, total **£7.29**. Grace agrees.

Fixed! …and a test failed?

```
$ uv run pytest -m smoke
FAILED tests/unit/pricing/test_calculator.py::test_loyalty_discount_applied_once
[XPASS(strict)] ORD-317: remove when fixed
1 failed, 17 passed
```

> We fixed the bug and a test failed. Welcome to software engineering.

Dan wrote a test that **expected** the bug (`xfail`, `strict=True`), so it fails loudly the moment someone fixes it. Why teams do this: **Lecture 5**.

Friday, 16:30

> **From:** Jordan
>
> Brilliant first week: ORD-317 fixed, `calc` finally has a name.
>
> One thing: all of it is still **on your laptop**. On this team, nothing reaches customers without **tests**, a **review** and **CI**. Dan's question stands: how do we make sure nobody breaks it again?

Lecture 5: CI/CD: tests, Git and pull requests, code review, GitHub Actions.

Before next week

- **Exercise ORD-318:** the student discount vanishes on Fridays. Debugger only, no `print()`!
- **Lecture 3 labs**, each with a self-check: **Beginner** (do the lecture yourself) · **Intermediate** (new bugs, fewer hints) · **Advanced** (the auditors are coming)
- **Profiling (Demo 4)**, not covered live: Grace's slow month-end report, `cProfile` and flame graphs
- Every step is written up in `lectures/03-ide-debug-large-code-bases/demo/`

Two short forms, please (about 8 minutes)

1. Attendance and survey

University sign-in · about 5 minutes

2. Anonymous feedback

No names recorded · about 2 minutes

Can't scan? Both links are on Moodle. Taking part in the research is **voluntary** and doesn't affect your place on the programme. Every question has an **N/A** option.

Thank you

Questions?

Remember: `print("here")` is a cry for help. Use a breakpoint.